

Bu ödevde 32 bit RISC-V buyruk kümesi mimarisi için bazı temel buyrukları ve verdiğimiz özel buyrukları gerçekleyeceksiniz. Bu özel buyrukları LLVM derleyici kütüphanesine RISC-V eklentisi olarak ekleyip daha sonra bu buyrukları içeren programları derleyip tasarlayacağınız özel buyruk eklentili basit bir işlemci üzerinde çalıştıracaksınız. Tablo 1’de RISC-V buyruk kümesinden kullanacağınız bir kısım temel buyruklar, eklenmesi gereken özel buyruklar ve buyruk kodlamaları görülebilir, özel buyrukların işlevleri ise Tablo 2’de açıklanmıştır.

TABLE 1: Temel ve Özel RISC-V Buyrukları

31 30 29		25 24	20 19	15 14	12 11	7 6	0	
funct7		rs2		rs1	funct3	rd	opcode	R-type
imm[11:0]				rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode		S-type
imm[12:10:5]		rs2	rs1	funct3	imm[4:1 11]	opcode		B-type
imm[31:12]					rd	opcode		U-type
imm[20:10:1 11 19:12]					rd	opcode		J-type
s1	funct11			rs1	funct3	rd	opcode	Özel-tip1
s2	imm[9:5]	rs2	rs1	funct3	imm[4:1 10]	opcode		Özel-tip2

Kullanmanız Gereken RISC-V Temel Buyrukları

imm[31:12]				rd	0110111	LUI
imm[31:12]				rd	0010111	AUIPC
imm[20 10:1 11 19:12]				rd	1101111	JAL
imm[11:0]		rs1	000	rd	1100111	JALR
imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011	BNE
imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011	BLT
imm[11:0]		rs1	010	rd	0000011	LW
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW
0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	100	rd	0110011	XOR
0100000	rs2	rs1	101	rd	0110011	SRA
imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	010	rd	0010011	SLTI
0000000	imm[4:0]	rs1	101	rd	0010011	SRLI
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	011	rd	0110011	SLTU

Özel Buyruklar

imm[11:0]		rs1	011	rd	1110111	DEGIL.YUKLE	
0100010		rs2	rs1	101	rd	1110111	YUK.VE.YAZ
imm[20:10:1 11 19:12]				rd	1111111	DORTKAT.ATLA	
s1	00101010101		rs1	010	rd	1110111	BIT.CEVIR
s2	imm[9:5]	rs2	rs1	111	imm[4:1 10]	1110111	SINIR.KNT

TABLE 2: Özel Buyrukların İşlevleri

Buyruk	Buyruğun İşlevi ve Formatı
DEGIL.YUKLE	<i>İşaretle genişletilmiş anlık değerin</i> bitlerinin değili (not) alınmış halini <i>rd</i> yazmacına yazar. (<i>rs1</i> RISC-V'de her zaman 0 olması gereken <i>x0</i> yazmacıdır.) Assembly formatı: <i>degil.yukle rd, rs1, imm</i>
YUK.VE.YAZ	<i>rs1</i> yazmacında bulunan adresteki veriyi veri belleğinden okur, <i>rs2</i> yazmacındaki değer ile her bit için bitsel ve (bitwise and) işlemi yapar ve oluşan değeri yine <i>rs1</i> yazmacında bulunan veri belleği adresine yazar. Assembly formatı: <i>yuk.ve.yaz rd, rs1, rs2</i>
DORTKAT.ATLA	<i>İşaretle genişletilmiş anlık değerin</i> 4 katına atlar (<i>program sayacını</i> günceller) ve <i>program sayacının</i> bir sonraki buyruk için olan değerini (<i>ps + 4</i>) <i>rd</i> yazmacına kaydeder. (ayrıca <i>imm[0] = 0</i>) Assembly formatı: <i>dortkat.atla rd, imm</i>
BIT.CEVIR	<i>s1 (1 bitlik seçim)</i> bitine bakarak <i>rs1</i> yazmacındaki değeri ya da bitlerin tersini <i>rd</i> yazmacına yazar. Seçim biti "1" ise bitlerin sırasını tersine çevirerek, "0" ise değerin kendisini yazar. Assembly formatı: <i>bit.cevir rd, rs1, s1</i>
SINIR.KNT	Veri belleğinden <i>anlık değer (imm)</i> adresindeki değeri okur ve <i>s2 (2 bitlik seçim)</i> bitlerine bakarak bu değeri karşılaştırır; <i>s2 = 00</i> iken <i>rs1 < veri < rs2</i> ise 1 değerini, değilse 0 değerini <i>rs1</i> yazmacına yazar. <i>s2 = 01</i> iken <i>rs1 > veri > rs2</i> ise 1 değerini, değilse 0 değerini <i>rs2</i> yazmacına yazar. <i>s2 = 10</i> iken <i>rs1 == veri</i> ise 1 değerini, değilse 0 değerini <i>rs2</i> yazmacına yazar. <i>s2 = 11</i> iken <i>rs2 == veri</i> ise 1 değerini, değilse 0 değerini <i>rs1</i> yazmacına yazar. (ayrıca <i>imm[0] = 0</i>) Assembly formatı: <i>sinir.knt rs1, rs2, imm, s2</i>

Uygulayacağınız RISC-V temel buyrukları için aşağıdaki dokümanları kontrol edebilirsiniz:

https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf

<https://msyksphinz-self.github.io/riscv-isadoc>

[40 Puan] RISC-V Buyruk Kümesi LLVM Eklentisi

LLVM, birçok frontend dilden (C/C++, Swift, Rust, Julia vb.) ortak bir ara dile (LLVM IR) çevrim yapan ve bu ara dilden de birçok backende (x86, ARM, MIPS, RISC-V vb.) özel kod üretebilen modüler bir derleyici kütüphanesidir. (<https://llvm.org>) Siz de bu ödevde, LLVM'in sağladığı bu esnekliği kullanarak verilen özel buyrukları makine koduna çevirebilmek için LLVM'in RISC-V backendine, standart I, M, C, A, F, D, Q eklentileri dışında bir buyruk eklentisi ekleyeceksiniz. Buyruk tanımlamaları için *LLVM Target Description Language* kullanacaksınız.

Eklentiye eklemekten önce gerekli bağımlılıkları yüklemeniz ve kütüphaneyi doğru bir şekilde build etmeniz gerek.

Not: Windows kullanıyorsanız Ubuntu WSL (<https://ubuntu.com/wsl>) ya da VirtualBox (<https://www.virtualbox.org>) gibi bir program üzerinde sanal makine kullanmanızı ve ubuntu için olan adımları takip etmenizi tavsiye ederim.

cmake ve *ninja* build araçlarını, ayrıca yoksa *gcc*, *g++* derleyicilerini ve *git*i, linux (örn. ubuntu) kullanıyorsanız terminali açın ve aşağıdaki gibi yükleyin:

```
sudo apt update
sudo snap refresh
sudo apt install gcc g++
sudo apt install git
sudo snap install cmake --classic
sudo apt install ninja-build
```

macOS kullanıyorsanız aşağıdaki gibi yükleyin:

```
brew update
brew install gcc
brew install git
brew install cmake
brew install ninja
```

Not: Bundan sonraki adımlar linux ve macOS sistemler için farketmiyor, aynı komutları kullanın.

LLVM kütüphanesini github'dan klonlayın ve 461274b81d8641eab64d494accddc81d7db8a09e commit versiyonuna getirin (LLVM 18.1.0 versiyonu) (<https://github.com/llvm/llvm-project/archive/refs/tags/llvmorg-18.1.0.zip> adresinden de indirebilirsiniz):

```
git clone https://github.com/llvm/llvm-project
cd llvm-project
git checkout 461274b81d8641eab64d494accddc81d7db8a09e
```

Kütüphaneyi elde ettikten sonra aşağıdaki gibi bir *build* klasörü oluşturun ve bu klasörün içine gerekli araçları;

clang → C kodunu derlemek için

llc → LLVM IR veya LLVM bytecode ara dilini derlemek için

llvm-objdump → object koddan hex kodu çıktısını almak ve assembly karşılıklarını görmek için

RISC-V backendi için verilen konfigürasyonlar ile build edin:

```
cd llvm-project
mkdir build
cd build
export CXXFLAGS="-std=c++11 -include limits"
cmake -G Ninja \
-DMAKE_BUILD_TYPE=Release \
-DLLVM_ENABLE_PROJECTS=clang \
-DLLVM_TARGETS_TO_BUILD=RISCV \
-DBUILD_SHARED_LIBS=True \
-DLLVM_PARALLEL_LINK_JOBS=1 \
../llvm
ninja -j1 clang llc llvm-objdump
```

Bu şekilde, ihtiyacınız olan *clang*, *llc* ve *llvm-objdump* programları *llvm-project/build/bin* dosya yolunda oluşmuş oldu. Burada daha hızlı build etmek için *ninja -j1* ifadesinde "1" yerine daha fazla iş parçacığı (thread) atayabilirsiniz. Maksimum atayabileceğiniz iş parçacığı sayısını, terminalde *nproc* komutu ile öğrenebilirsiniz.

Eğer kütüphane sorunsuz olarak build edildiyse bundan sonra eklentiye ekleme işlemine geçebilirsiniz.

Eklenti eklemek için dosyalarda yapmanız gereken değişiklikler aşağıda anlatılmıştır. Eklenti ismi Türkçe karakter olmadan soyadınız olsun. Burada örnek olarak "kasirga" kullanılmıştır.

Buyruk eklentisini eklemeniz için gereken dosya değişiklikleri **llvm-project/llvm/lib/Target/RISCV** dosya yolunda yapılmalıdır. LLVM kütüphanesinde **llvm-project/llvm/lib/Target/RISCV** dosya yoluna gidin ve eklentideki buyrukları tanımlayacağınız **RISCVInstrInfo<Soyad>.td** dosyasını oluşturun:

```
cd llvm-project/llvm/lib/Target/RISCV
touch RISCVInstrInfoKasirga.td
```

RISCVInstrInfo.td dosyasının sonunda, oluşturduğunuz **RISCVInstrInfo<Soyad>.td** dosyasını aşağıdaki gibi bir satır ekleyerek dahil edin:

```
include "RISCVInstrInfoKasirga.td"
```

RISCVFeatures.td dosyasında, eklentiye aşağıdaki gibi tanımlayın (soyadınızı eklemek için değiştirmeniz gereken yerler kırmızı renk ile gösterilmiştir):

```
def FeatureExtKasirga
: SubtargetFeature<"kasirga", "HasExtKasirga", "true",
  "'K' (Kasirga Buyruklari)">;
def HasExtKasirga : Predicate<"Subtarget->hasExtKasirga()">,
  AssemblerPredicate<(all_of FeatureExtKasirga),
  "'K' (Kasirga Buyruklari)">;
```

Bundan sonra **RISCVInstrInfo<Soyad>.td** dosyasına özel buyrukları ekleyebilirsiniz. *0000000_rs2_rs1_000_rd_0000000* kodlamasına sahip örnek bir buyruğu aşağıdaki gibi ekleyebilirsiniz:

```
let hasSideEffects = 0, mayLoad = 0, mayStore = 0 in
def ORNEK_BUYRUK : RVInst<(outs GPR:$rd), (ins GPR:$rs1, GPR:$rs2),
  "ornek.buyruk", "$rd, $rs1, $rs2", [], InstFormatOther>, Sched<[]>
{
  bits<5> rs2;
  bits<5> rs1;
  bits<5> rd;

  let Inst{31-25} = 0b00000000;
  let Inst{24-20} = rs2;
  let Inst{19-15} = rs1;
  let Inst{14-12} = 0b000;
  let Inst{11-7} = rd;
  let Inst{6-0} = 0b00000000;
}
```

Burada anlık değeri olan buyrukların anlık değerleri için *GPR* yazmaç tanımlaması yerine, *simm12*, *simm13_lsb0*, *simm21_lsb0_jal* gibi tanımlamalar kullanmalısınız. Diğer buyruk yapılarını örnekte gösterildiği gibi, **RISCVInstInfo<Ekleni_İsmi>.td** dosyalarında benzer buyrukların nasıl çözüldüğüne bakarak ya da internette araştırarak - deneyerek ekleyebilirsiniz.

Buyrukları ekledikten sonra değişikliklerin olması için **llvm-project/build** klasörüne giderek kütüphaneyi aynı şekilde tekrar build etmeniz gerekiyor:

```
cd llvm-project/build
ninja -j1 clang llc llvm-objdump
```

Bundan sonra eklediğiniz buyrukları içeren programları *clang* ve *llc* ile derleyebileceksiniz ve *llvm-objdump* ile, derlenmiş programlardaki *hex (on altılık taban) kodunu* görebileceksiniz.

Kütüphaneyi tekrar build ettikten sonra aşağıdaki *inline assembly* (<https://en.cppreference.com/w/c/language/asm>) olarak yazılmış C kodunu bir dosyaya kaydedin, *clang* ve *llc* ile programı derleyin ve *llvm-objdump* ile çıktıya bakın:

```
// ornek.c
asm ("lui x5, 0x2");
asm ("addi x6, x5, 512");
asm ("auipc x7, 0");
asm ("degil.yukle x8, x0, 2046");
asm ("ori x9, x8, 255");
asm ("sw x9, 0(x6)");
asm ("lw x10, 0(x6)");
asm ("add x11, x10, x7");
asm ("sub x12, x11, x5");
asm ("xor x13, x12, x9");
asm ("sra x14, x13, x10");
asm ("srli x15, x14, 2");
asm ("slt x16, x15, x12");
asm ("sltu x17, x16, x11");
asm ("slti x18, x17, 8");
asm ("bit.cevir x19, x18, 1");
asm ("yuk.ve.yaz x6, x6, x19");
asm ("bne x16, x17, 0x00000200");
asm ("blt x18, x19, 0x00000800");
asm ("dortkat.atla x20, 0x00000080");
asm ("0x00000200:\n"
    "sinir.knt x18, x17, 0x10, 1\n"
    "jalr x21, 0(x5)");
asm ("0x00002000:\n"
    "sinir.knt x22, x17, 0x20, 3\n"
    "jal x1, 0x00000800");
asm ("0x00000800:\n"
    "jal x1, 0x00000800");
```

Not: Örnek C programı inline assembly olarak yazıldı fakat işlemcinizde bulunan RISC-V buyruklarını içeren (derlendiği zaman) normal bir C kodunu da işlemcide çalıştırabilmelisiniz. (İşlemcinizde RISC-V temel buyruklarının bir alt kümesi istendiği için her program çalışmayacaktır.)

Derlerken -mattr flagi +<soyad> şeklinde olmalı:

```
## C kodunu RISC-V Assembly koduna derleme - gerekli değil - sadece kontrol için
llvm-project/build/bin/clang -S -target riscv32 -O0 ornek.c -o ornek.s

## C kodunu LLVM IR koduna derleme
llvm-project/build/bin/clang -S -emit-llvm -target riscv32 -O0 ornek.c -o ornek.ll

## LLVM IR kodunu object koda derleme
llvm-project/build/bin/llc -mtriple=riscv32 -O0 -mattr=+kasirga -filetype=obj \
ornek.ll -o ornek.o

## Object koddan hex çıktısını alma
llvm-project/build/bin/llvm-objdump -d ornek.o
```

Örnek olarak oluşan *llvm-objdump* çıktısı aşağıdaki gibidir (Kırmızı renkli kısım ilgili buyruğun hex (on altılık taban) kodu, yeşil renkli kısım ise denk gelen buyrukların assembly karşılığını göstermektedir. Assembly kısmında *0x* ile yazılmayan sayılar ondalık (decimal) tabandadır.):

```
shc@kasirga:~/projects$ llvm-project/build/bin/llvm-objdump -d ornek2.o
```

```
ornek2.o:      file format elf32-littleriscv
```

```
Disassembly of section .text:
```

```
00000000 <.text>:
```

```
0: b7 22 00 00 lui    t0, 0x2
4: 13 83 02 20 addi   t1, t0, 0x200
8: 97 03 00 00 auipc  t2, 0x0
c: 77 34 e0 7f degil.yukle    s0, zero, 0x7fe
10: 93 64 f4 0f ori    s1, s0, 0xff
14: 23 20 93 00 sw     s1, 0x0(t1)
18: 03 25 03 00 lw     a0, 0x0(t1)
1c: b3 05 75 00 add    a1, a0, t2
20: 33 86 55 40 sub    a2, a1, t0
24: b3 46 96 00 xor    a3, a2, s1
28: 33 d7 a6 40 sra    a4, a3, a0
2c: 93 57 27 00 srli   a5, a4, 0x2
30: 33 a8 c7 00 slt    a6, a5, a2
34: b3 38 b8 00 sltu   a7, a6, a1
38: 13 a9 88 00 slti   s2, a7, 0x8
3c: f7 29 59 95 bit.cevir    s3, s2, 0x1
40: 77 53 33 45 yuk.ve.yaz    t1, t1, s3
44: 63 10 18 21 bne    a6, a7, 0x244 <.text+0x244>
48: e3 40 39 01 blt    s2, s3, 0x848 <.text+0x848>
4c: 7f 0a 00 08 dortkat.atla    s4, 0x80
50: 77 78 19 41 sinir.knt      s2, a7, 0x10, 0x1
54: e7 8a 02 00 jalr    s5, t0
58: 77 70 1b c3 sinir.knt      s6, a7, 0x20, 0x3
5c: ef 00 10 00 jal     0x85c <.text+0x85c>
60: ef 00 10 00 jal     0x860 <.text+0x860>
```

Buyrukları ekledikten sonra sizin derleyicinizde bu örnek program için hex kodunun bu şekilde gelip gelmediğini kontrol edebilirsiniz. Buyrukların hex (on altılık taban) kodunun küçüğü başta ve bayt adreslemeye uygun bayt bayt (8 bit 8 bit) geldiğini görüyorsunuz. Örneğin *degil.yukle* buyruğunun hex kodu **77_34_e0_7f** şeklinde fakat aslında **7f_e0_34_77** olarak çevirmelisiniz, bu şekilde doğru binary karşılığı olan **01111111110_00000_011_01000_1110111** karşılık geldiğini göreceksiniz.

Bu programı 2. bölümde tasarlayacağınız işlemci üzerinde çalıştırarak işlemcinizi deneyebilirsiniz.

[60 Puan] RISC-V Buyruk Kümesi Eklentili İşlemci Verilog Tasarımı

Verilen buyrukları gerçekleyen tek vuruşlu 32 bit işlemciyi Verilog donanım tanımlama dilinde davranışsal modelleme kullanarak yazın. Dosya ismi küçük harflerle ve Türkçe karakter olmadan <soyad>.v şeklinde soyadınız olsun. "<soyad>" modülünün giriş çıkışları Tablo 3'te görülebilir.

TABLE 3: <soyad> Modülünün Giriş ve Çıkış Sinyalleri

Sinyal	Yön	Genişlik	Açıklama
saat	Giriş	1 bit	Sıralı mantık için gerekli saat girişi.
reset	Giriş	1 bit	Devreyi başlangıç durumuna döndüren sinyal.
buyruk	Giriş	32 bit	İşlemcinin şu anki saat vuruşunda yürüteceği buyruk.
program_sayaci	Çıkış	32 bit	Bir sonraki çevrim getirilecek buyruğun adresi.
yazmaclar	Çıkış	32 x 32 bit (1024 bit) {x31, ..., x0}	32 bit genişliğe sahip 32 adet değerlerin saklandığı yazmaçlar.
vb_yaz	Çıkış	1 bit	Veri belleğine yazmak için etkinleştirme sinyali.
vb_adres	Çıkış	32 bit	Veri belleğine erişimde için kullanılacak adres.
vb_yaz_veri	Çıkış	32 bit	Veri belleğine yazılacak veri.
vb_oku_veri	Giriş	32 bit	Veri belleğinden okunan veri.

İşlemciniz saatin pozitif kenarında, her saat vuruşunda *buyruk* sinyali üzerinden gelen komutu çözmeli, yazmaç değerlerini ve program sayacını güncellemeli ve varsa bellek işlemlerini gerçekleştirmelidir. Devre sıfırlandıktan sonra, program sayacı 0x0 konumundayken ilk komutu almalıdır. Belirtildiği gibi, hex çıktısında buyruklar küçüğü başta olduğundan gelen buyruklar önce uygun formata çevrildikten sonra çözülmelidir. Veri belleğinden okumalar kombinasyonel iken yazmalar ardışıktır, yani, okumalar anlık (0 çevrim) yapılabilirken veri yazma 1 çevrimde gerçekleşir. Veri belleği ve buyrukların yüklenmesi testbench tarafından yapılacaktır.

Ödev Teslimi (Son Teslim Tarihi: 15.03.2026 23.59)

⚠ Çevrimiçi kaynakları veya üretken yapay zeka araçlarını kullanabilirsiniz, ancak tüm kodlarda intihal kontrolü yapılacak ve eşleşmeler -yapay zeka ile yapıp yapılmamasından bağımsız olarak- ödevden 0 puan alacaktır. Eğer kendiniz yaparsanız -yapay zekanın yardımıyla bile olsa- endişelenmenize gerek yok.

1-) RISCFeatures.td

2-) RISCInstrInfo.td

3-) RISCInstrInfo<Surname>.td

4-) <surname>.v

dosyalarını sıkıştırın ve <Soyad_Öğrenci Numarası>.zip olarak kaydederek <https://uzak.etu.edu.tr>'ye yükleyin.